The Making of LabBench

An engineering build-log: five interactive tools, the code behind each one, the mistakes I made, and how a set of demos became an actual product.

by Dhananjay Kumar Seth
labbench-hub.vercel.app

Why this exists

Most engineering concepts live in textbooks and MATLAB scripts. You read about a Fourier transform, a PID loop, or a constellation diagram, but you rarely get to *touch* one. I'm a game developer by day and an Electronics & Communication Engineer by training, and I kept thinking: why can't these just be things you open in a browser and play with?

So I built five of them, hand-wrote every piece of math myself, and then turned the whole thing into LabBench — a real, licensed, monetized product line. This document is the part I didn't put in the public write-up: the actual code, the bugs that bit me, and the non-technical decisions (branding, licensing, payments) that took the demos from "cool side project" to "thing people can pay for."

Contents

1. DSP Signal Lab

The key realization: the browser already has a real FFT built in. The Web Audio API's `AnalyserNode` performs an actual Fast Fourier Transform on any audio flowing through it — you don't implement the transform, you just read the result every frame.

```
const ctx = new AudioContext();
const analyser = ctx.createAnalyser();
analyser.fftSize = 2048;
const freqData = new Uint8Array(analyser.frequencyBinCount);
const timeData = new Float32Array(analyser.fftSize);
```

Filters came free too — a `BiquadFilterNode` gives lowpass / highpass / bandpass / notch with adjustable cutoff and Q in one node, chainable directly into the signal path:

```
const filter = ctx.createBiquadFilter();
filter.type = 'lowpass';
filter.frequency.value = 1000;
osc.connect(filter).connect(analyser).connect(ctx.destination);
```

> ***Lesson learned:*** *React's StrictMode intentionally double-invokes effects in development. For a component that builds an imperative Web Audio graph on mount, that meant two oscillators, two filters, and — worst of all — a phantom microphone permission prompt on a page that hadn't even touched the mic yet. Dropping StrictMode is a known, accepted exception for imperative audio/canvas apps; I do it in every LabBench tool now.*

2. PID Control Playground

The robot isn't just a slider hooked to a rotation value — it's a genuine double-integrator plant, sub-stepped so the integral term doesn't blow up at low frame rates:

```
const sub = 3, h = dt / sub;
for (let k = 0; k < sub; k++) {
  const e = y - ref;
  integral += e * h;
  integral = clamp(integral, -260, 260);       // anti-windup
  const dedt = (e - lastE) / h;
  const u = kp*e + ki*integral + kd*dedt;
  vy += (-u - 0.9*vy) * h;                     // damped double integrator
  y += vy * h;
}
```

The anti-windup clamp on `integral` matters more than it looks — without it, a saturated integral term causes wild overshoot the instant the robot re-engages the track after a disturbance.

> ***Lesson learned:*** *Sub-stepping the physics (running the integrator 3x per animation frame instead of once) turned out to be the difference between a controller that looks like it's fighting a washing machine and one that looks like real hardware. Animation frame rate and control-loop stability are not the same clock, and conflating them is a classic beginner mistake in interactive simulations.*

3. Logic Circuit Simulator

A naive left-to-right circuit evaluator works fine for combinational logic, but it deadlocks the instant you build something with feedback — an SR latch, for instance, where each NOR gate's output

feeds the other gate's input. There's no valid "order" to evaluate them in.

The fix is an iterative relaxation solver: evaluate every gate once per pass using whatever values are currently known, repeat, and stop once nothing changes (or a pass limit is hit):

```
for (let iter = 0; iter < 40; iter++) {
  const next = {};
  for (const c of comps) next[c.id] = evaluate(c, val);
  const changed = comps.some(c => next[c.id] !== val[c.id]);
  Object.assign(val, next);
  if (!changed) break;
}
```

> ***Lesson learned:*** *40 passes is overkill for any circuit a human would build by hand in this tool, but the cost of extra passes is negligible and the cost of an under-converged latch is a visibly broken demo. When the failure mode is embarrassing and the fix is cheap, over-provision.*

4. Comms Simulator

The most satisfying bug-hunt in the whole project: getting the Monte-Carlo bit-error-rate curve to actually hug the theoretical Q-function curve. Every noise sample uses a proper Box-Muller transform (not `Math.random()` alone, which is uniform, not Gaussian):

```
function gauss() {
  const u = Math.random(), v = Math.random();
  return Math.sqrt(-2 * Math.log(u)) * Math.cos(2 * Math.PI * v);
}
```

The theoretical curve comes from the Gaussian Q-function, approximated via the Abramowitz-Stegun erfc series (accurate to about 1.5e-7, plenty for a visual match). Simulated and theoretical curves should converge as bits-per-point goes up — and watching them actually line up on screen after getting the noise model right was the single best debugging payoff of the project.

> ***Lesson learned:*** *The BER floor at very low error rates (around 1e-5 with 60,000 simulated bits) isn't a bug — it's the Monte-Carlo method telling the truth. You cannot resolve an error rate rarer than roughly 1/N with N trials. I nearly "fixed" this floor before realizing it was mathematically correct behavior, not a defect.*

5. Waveform Viewer

The newest tool in the suite: a Verilog-style timing-diagram editor. Click a cell on the D input row and every derived register recomputes, modeled as genuine positive-edge-triggered logic — a register changes exactly one clock period after the edge that caused it:

```
// D flip-flop: Q captures D from the PREVIOUS period
const q = Array(n).fill(0);
for (let i = 1; i < n; i++) q[i] = control[i - 1];
```

The 4-bit counter is the same idea generalized — each bit is a divide-by-2 of the one before it, which is why the classic binary ripple pattern (Q0 toggles every cycle, Q1 every 2, Q2 every 4...) falls out for free once the D flip-flop model is correct.

> ***Lesson learned:*** *Testing an interactive click-to-toggle SVG with an automated browser tool taught me something concrete: a raw JavaScript* `element.dispatchEvent(new MouseEvent('click'))`

does not reliably trigger React's synthetic event handlers the way a real user click does. Real synthetic input (via an actual browser automation click) worked every time; manually dispatched events silently did nothing. Worth knowing before you assume your automated test actually clicked anything.

6. From demos to a product

Five working tools are not a product. This is the part most "I built X" posts skip, and it's arguably the more useful half for anyone trying to do the same thing.

6.1 — Picking a brand, not just a project name

Four separate demo names ("dsp-signal-lab", "pid-control-playground"...) don't add up to anything a stranger remembers. I picked one umbrella name — **LabBench** — built a single landing page presenting all five tools as one suite, and cross-linked every individual tool back to it with a small badge. The individual tools keep their own visual identity (each has its own accent color); the brand sits one layer above that as the thing people actually remember and share.

6.2 — Licensing: the part everyone gets wrong

None of the five repos had a LICENSE file at first. That sounds harmless, but the default in every jurisdiction under the Berne Convention is "all rights reserved" — meaning, ironically, nothing was actually more locked-down than an unlicensed public repo. The fix wasn't to slap on MIT, though. An MIT license would let anyone fork the free tools, add the exact Pro features I'm planning, and resell them before I ship mine.

Instead I wrote a short custom **source-available** license: free to use the deployed demo, free to read the code for learning, explicitly *not* free to redistribute, resell, or rehost. That's the license every LabBench repo carries now.

6.3 — Monetization: the freemium ladder

- **Tip jar** — the fastest thing to ship. Not a serious revenue line, but it's live today and costs nothing to maintain.
- **Digital products** — this very document, and a companion study guide, sold as one-time downloads to the same audience already using the free tools.
- **Pro tier (live)** — saved sessions across all five tools, plus PNG/SVG/VCD export on Waveform Viewer, gated behind real accounts and a Razorpay subscription. The free tools stay free forever; Pro adds capability on top rather than paywalling what already works. See chapter 7 for how it's actually built.
- **Institutional licensing** — the highest-ceiling option and the one that needs no new code: colleges and coaching institutes have actual budgets, unlike individual students, and a flat classroom-license deal can be sold manually before any of the automation above exists.

6.4 — The payments problem nobody warns you about

If you're building from India (or several other countries), the "just add Stripe / Ko-fi / Buy Me a Coffee" advice you'll find everywhere quietly assumes a US or EU creator. Ko-fi's payout options for India returned zero results for a bank-transfer country search; Buy Me a Coffee's own FAQ confirms payouts route through a personal Stripe account, and Stripe Connect payouts to individual Indian creators are heavily restricted without a registered business entity.

The fix was to stop chasing Western platforms and go regionally native: **Razorpay**, an Indian payment processor, supports individual/freelancer accounts directly, with UPI, cards, and netbanking all working out of the box once KYC clears. If you're building a product with a specific home country's audience, check the payout path for creators in *that* country before you build anything around a specific platform — the "best" tipping platform by download count is not universal.

7. Building the Pro tier

"Saved sessions" sounds like one feature. It's actually three separate systems that have to agree with each other: an identity system (who is this?), a payment system (did they pay?), and a data system (what did they save?) — and getting the boundaries between those three right mattered more than any individual line of code.

7.1 — One account, five tools, one Supabase project

Each LabBench tool is a separate deployment on a separate domain. The tempting mistake is to think that means five separate user systems. It doesn't have to: all five tools point at the *same* Supabase project (same URL, same publishable key), so a user who signs up on one tool has a real account the moment they sign in anywhere else in the suite — because there is only one `auth.users` table, not five.

```
// identical in every tool's src/supabaseClient.ts
const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
);
```

The Pro-vs-free flag (`profiles.is_pro`) lives in that same shared project, keyed by user_id. Which means a single subscription genuinely unlocks Pro everywhere, without a cross-service sync job of any kind — it's not five accounts pretending to be linked, it's actually one account.

> ***Lesson learned:*** *Sessions don't share across origins even when the backend does. Signing in on dsp-signal-lab.vercel.app doesn't sign you in on comms-simulator.vercel.app, because Supabase Auth stores its session in localStorage, which is per-origin by browser design. Every tool still needs its own sign-in UI; only the account and the Pro flag are actually shared.*

7.2 — Only one tool talks to Razorpay

Five tools could each have their own checkout code. They don't. Only Logic Circuit Simulator runs the Razorpay Subscriptions integration (`create-subscription.js` and `subscription-webhook.js`) — the other four just show an "Upgrade on LabBench" link that sends the user there. One place holding the payment secrets and webhook logic is easier to audit and reason about than five copies drifting out of sync, and since the Pro flag is shared, it doesn't matter which tool the user actually upgraded from.

```
// api/create-subscription.js (server-side only)
const userRes = await fetch(`${SUPABASE_URL}/auth/v1/user`, {
  headers: { Authorization: `Bearer ${token}`, apikey: SUPABASE_ANON_KEY },
});
const user = await userRes.json();  // never trust a client-sent user_id

await fetch('https://api.razorpay.com/v1/subscriptions', {
  method: 'POST',
  headers: { Authorization: 'Basic ' + base64(KEY_ID + ':' + KEY_SECRET) },
  body: JSON.stringify({ plan_id: PLAN_ID, notes: { user_id: user.id } }),
});
```

That `notes.user_id` is the thread that ties a Razorpay subscription back to a specific Supabase account. When Razorpay later calls the webhook, that's the only piece of information the webhook

actually trusts.

7.3 — Verifying a webhook you didn't ask for

A webhook endpoint is a URL the whole internet can POST to. The only thing separating a real Razorpay event from a stranger's forged JSON is the HMAC signature in the request header, checked with a constant-time comparison so a timing attack can't leak the correct signature byte by byte:

```
const expected = crypto.createHmac('sha256', WEBHOOK_SECRET)
  .update(rawBody).digest('hex');
const valid = header.length === expected.length &&
  crypto.timingSafeEqual(Buffer.from(header), Buffer.from(expected));
if (!valid) return res.status(403).json({ error: 'invalid signature' });
```

> ***Lesson learned:*** *The very first test of this webhook used a made-up UUID for user_id and came back as a Postgres foreign-key violation, not a success. That was actually the best possible result: it proved the signature check, the Supabase write, and the database's own referential-integrity guard were all working, using a failure instead of a success to confirm three systems at once.*

7.4 — Two more "reliable-looking" browser APIs that aren't

Both bugs below share a pattern: the code looks obviously correct, runs fine for a human clicking slowly, and only breaks under automated testing or on certain mobile browsers.

```
// looks fine, silently breaks:
const fn = mode === 'up' ? supabase.auth.signUp : supabase.auth.signInWithPassword;
await fn({ email, password });   // 'this' is now undefined inside fn

// correct:
await (mode === 'up'
  ? supabase.auth.signUp({ email, password })
  : supabase.auth.signInWithPassword({ email, password }));
```

Extracting a method off an object as a bare reference drops the `this` binding it needs internally. The call fails before it ever reaches the network — no error, no request, a button stuck on "..." forever. Separately, `window.prompt()` and `window.confirm()` — used for naming a saved session and confirming a delete — turned out to be unreliable enough (silently auto-dismissed in some automated contexts, and observed to fully freeze the render thread in others) that both got replaced with plain inline React UI: a text input for naming, and a two-click "tap once to arm, tap again to confirm" pattern for delete.

> ***Lesson learned:*** *Native browser dialogs are a trap for anything you actually ship. They're fine for a quick internal tool; for a real feature, build the confirmation into your own UI from the start and skip re-learning this the hard way.*

7.5 — The account-activation wall

The code side of Razorpay Subscriptions worked on the first real attempt — the subscription got created, Checkout opened, the test card was accepted right up to the final authorization step, which then failed with "the seller does not support recurring payments." That error has nothing to do with application code: recurring payments (UPI Autopay / e-NACH / card auto-debit) require the Razorpay account itself to be fully KYC-activated, not just capable of taking one-time Payment Page payments, which apparently needs a lighter bar.

> ***Lesson learned:*** *When a payment gateway integration fails at the very last step with a vague seller-side message, check account activation/KYC status before debugging your own code further. A perfectly correct integration will still fail if the underlying merchant account isn't cleared for that specific payment type.*

Closing

None of this required outside funding, a team, or permission from anyone. Five tools, one brand, a license that protects the future paid tier, and a payment rail that actually works for where I'm building from. The tools themselves are still free — go use them.

labbench-hub.vercel.app

Thank you for supporting LabBench.